distributed monolith. I don't consider this a good name because it implies that monoliths are made up of tightly coupled, non-cohesive components by definition. That's not true. The architectural style doesn't matter: a bad design is a bad design. Indeed, I like the *modular monolith* term proposed by Simon Brown to increase awareness that monoliths can promote loose coupling and high cohesion, and that both monoliths and microservices can end up being "big balls of mud."

Throughout the book, I'll discuss some techniques for enforcing loose coupling in applications. In particular, we'll adopt a service-based architecture and focus on building services with clear interfaces to communicate with each other, minimal dependencies on other services, and high cohesion.

1.3.3 Resilience

A system is resilient if it provides its services even in the presence of faults or environmental changes. Resilience is "the capability of a hardware-software network to provide and maintain an acceptable level of service in the face of faults and challenges to normal operation."[7]

When building cloud native systems, our goal should be to guarantee that our applications are always available, whether there is a failure in the infrastructure or in our software. Cloud native applications run in a dynamic environment where everything keeps changing, and faults can and will occur. This cannot be prevented. In the past, we used to consider changes and faults as exceptions, but for highly distributed systems like cloud native ones, changes are not exceptions: they are the rule.

When discussing resilience, it's worth defining three essential concepts: fault, error, and failure:[8]